

INTERVIEW PREP · PHONE EDITION

Senior React Developer

NEOFACTO Luxembourg — financial-sector client

React · TypeScript · Jest + RTL

SPFx · SharePoint Online · Microsoft 365

Shared component libraries · A11y · Security

Prepared for Richard · July 2026

Senior level: trade-offs and reasoning, not syntax drills.

Contents

1. Game plan & who you're talking to · 3
2. Your story & the openers · 6
3. React — senior level · 8
4. TypeScript & modern JS · 14
5. Jest + React Testing Library · 17
6. SPFx & SharePoint crash course · 22
7. Architecture & shared libraries · 27
8. Security & accessibility · 29
9. Ways of working & STAR bank · 31
10. The gap questions — scripts · 34
11. The technical test stage · 36
12. Questions to ask them · 37
13. Last 48 hours · 38

1 · Game plan & who you're talking to

What the ad is really testing

- **React depth** — "deep knowledge of the React ecosystem and modern JavaScript" is the core bar. §3.
- **Testing** — Jest + React Testing Library is the only skill the ad calls *essential*. Expect concrete questions or a live exercise. §5.
- **SharePoint/SPFx** — only "some experience" is asked. This is the differentiator, not the entry bar; a focused half-day of hands-on ramp closes most of it. §6.
- **Enterprise discipline** — coding standards, security, accessibility, documentation, handover, dev/test/acceptance environments. A financial-sector client weighs governance as heavily as code. §8–9.
- **Communication** — explaining trade-offs to Tech Leads, BAs and non-technical stakeholders is an explicit requirement. Expect a scenario question. §9.

Intel on NEOFACTO

- Luxembourg software consultancy, founded 2000, ~80 people, office in Esch-Belval. Since September 2024 part of **Aricoma**, a Czech enterprise-IT group (KKCG) that serves EU institutions.
- Financial clients over the years have included the **European Investment Bank, ING, Spuerkeess, Foyer and the Luxembourg Stock Exchange**.
A "financial-sector client" needing SharePoint Online intranet work is very plausibly a bank or an EU institution: formal processes, security reviews, accessibility obligations.
- Fintech/blockchain heritage — the Scorechain crypto-compliance venture shares its founders. Culture per the ad: Great Place to Work, Neo'vations sessions, afterworks. Mention one of these once, naturally.
- **Process:** recruiter pre-qualification → technical tests → *salary proposal + exclusivity agreement* → business manager → submission to client → client interview. You commit to a number and exclusivity *before* meeting the client — have your figure ready from day one (§2, §12).

Your positioning — internalize this paragraph

Senior engineer, 16 years, with the recent Docler Luxembourg years spent on **production React + TypeScript** for a high-traffic platform: Figma-to-UI with designers and BAs, GraphQL data-layer work, feature-flagged rollouts, mentoring. Full-stack background, so integrating frontends with enterprise APIs is home turf. Your most recent hands-on work was **React 17** — which happens to be the exact version SPFx pins today (§6), so nothing about the platform's React is rusty for you. You live on the Luxembourg border and have already worked in this market: zero relocation friction. SharePoint is the gap; you close it honestly — ramp the fundamentals, build one demo web part before the interview, and run the "fast learner with proof" script (§10).

2 · Your story & the openers

The 2-minute pitch (structure, not a script)

1. **Now:** senior frontend/full-stack engineer, 16 years, most recently Docler Holding Luxembourg through December 2025.
2. **Recent proof:** production React/TS features on a high-traffic platform; translating Figma designs with designers and BAs; GraphQL integration cutting payloads ~30% on targeted flows; tooling that contributed to a ~10% smaller production bundle; Flagsmith feature flags for controlled rollouts; mentoring and internal tech talks.
3. **Depth:** earlier years building real-time Node.js backends at serious scale — you understand the systems your UIs call.
4. **Fit:** enterprise delivery habits (reviews, tests, documented handover), AWS certified, cross-border local to Luxembourg.
5. **Why this role:** back to deep frontend craft in a structured institutional environment — SharePoint as a platform to master, not a detour.

SAY IT LIKE THIS

"Why did you leave Docler?" — Two calm sentences: the position ended in December 2025
[adjust to your real framing — restructuring / end

of contract], and you've used the time since for certification-level upskilling, hands-on coding challenges and a selective search rather than grabbing the first offer. Zero negativity about Docler — interviewers read tone more than content here.

SAY IT LIKE THIS

"Why a consultancy after a product company?" — Variety of enterprise contexts, exposure to institutional clients, and at ~80 people your seniority is visible instead of diluted. Reference Neo'vations / the learning culture from the ad — it shows you actually read it.

PREPARE BEFORE THE FIRST CALL

Salary. Their process locks a salary proposal and an exclusivity agreement before you ever meet the client. Decide your gross annual target and your floor *before* the pre-qualification call; anchor with a range and tie the top to the financial-sector, on-site-in-Luxembourg constraints. Exclusivity means no other agency may submit you to this client — asking about its scope and duration is professional, not adversarial (§12).

3 · React — senior level

Answers as you'd actually give them: mechanism in one breath, then the judgement call. Depth on demand, not by default.

Walk me through what happens when state changes.

A state update schedules a re-render: React re-runs the component function, builds a new element tree, reconciles it against the previous one, and commits the minimal DOM mutations. Children re-render by default when a parent does — `React.memo`, stable references, or moving state down are what stop that. Since React 18, updates are batched everywhere; under React 17 — which SPFx runs today — batching only happens inside React event handlers, so two `setState` calls in a `setTimeout` mean two renders.

Why do keys matter, and when does index-as-key bite?

Keys give list items identity across renders so reconciliation can match, move and preserve them along with their state. Index keys break the moment the list reorders, inserts or deletes: state, focus and input values stay glued to

positions instead of items, producing wrong-row bugs and needless remounts. Use stable IDs from the data.

What did Fiber actually change? What is concurrent rendering?

Fiber turned rendering from one synchronous recursion into an interruptible series of work units, letting React pause low-priority rendering to service urgent input — the foundation for transitions, Suspense and `useDeferredValue`. The practical consequence: render functions must be pure, because React may run them and discard the result; side effects belong in effects and event handlers.

useEffect — mental model and the classic bug?

An effect synchronizes the component with an external system after commit; the dependency array declares what it reads. The classic bug is the stale closure — an interval reading state captured at mount — fixed with functional updates, a ref, or restructuring so the effect re-subscribes. Cleanup runs before the next effect

and on unmount. `useLayoutEffect` only for measuring or mutating the DOM before paint; it blocks painting, so it's the exception.

When do you reach for `useMemo` / `useCallback` / `React.memo`?

They're optimizations with a cost — memory, comparisons, reading noise — so: genuinely expensive computations, and referential stability for props crossing a `React.memo` boundary or feeding dependency arrays. I profile first with the React DevTools Profiler and memoize at boundaries, not everywhere. Newer React ships a compiler that automates this, but on a pinned-React platform like SPFx it remains manual discipline.

Context — when is it the wrong tool?

Every consumer re-renders when the value changes, so context suits low-frequency data — theme, auth, locale — and hurts with hot, fine-grained state. Mitigations: split state and dispatch into separate contexts, memoize provider values. Beyond that, an external store

with selector-based subscriptions (Zustand, Redux Toolkit, MobX — I shipped with MobX recently) scales better.

How do you decide where state lives?

Classify it first. Server state goes to a server-cache library (React Query / SWR) — caching, dedupe, retries and invalidation are solved problems I shouldn't hand-roll. UI state stays local, colocated as deep as possible. Genuinely cross-cutting client state gets one small store; shareable view state goes in the URL. The senior failure mode is one global store for everything.

Controlled vs uncontrolled — and large forms?

Controlled: React state is the source of truth — best for live validation and derived UI.

Uncontrolled: the DOM holds it, you read on submit. Fully controlled large forms re-render brutally, which is why React Hook Form — uncontrolled inputs plus targeted subscriptions — is my default at scale.

Error handling in a React page?

Error boundaries (class-based, or `react-error-boundary`) catch render and lifecycle errors and show a fallback; they don't catch event handlers or async code — those need try/catch and error state. I place granular boundaries around independent islands. On a SharePoint page that's not academic: one failing web part must not take down the whole intranet page.

Code splitting strategy?

`React.lazy` + `Suspense` at route and feature boundaries; dynamic `import()` for heavy, rarely used pieces like editors and charts. In SPFx every web part is its own bundle anyway — the interesting problem is shared code, so five web parts don't ship five copies (§6, §7).

React 17 vs 18 — what actually changes?

`createRoot`; automatic batching outside React events; the concurrent APIs (`startTransition`, `useDeferredValue`, `useId`, `useSyncExternalStore`); `StrictMode` double-invoking effects in dev to surface unsafe ones — libraries that depended on render timing needed updates. Why it matters here: SPFx still

pins React 17, with Microsoft signalling React 18 support around mid-2026 but no confirmed date — so I'd verify the tenant's SPFx baseline before promising anything modern. My latest hands-on project was React 17 + TypeScript, so exactly this version is fresh for me.

A page is slow. Walk me through it.

Reproduce and measure before touching code: Profiler for commit counts and render causes, the network tab for waterfalls. Fixes in order of cheapness: colocate state so fewer components re-render; memoize the real boundaries; virtualize long lists (`react-window`); split bundles; cache server data properly. Then measure again — performance work without numbers is storytelling.

Refs and imperative escape hatches?

Refs for DOM access and for mutable values that must survive renders without causing them. `forwardRef` + `useImperativeHandle` to expose a minimal imperative surface from library components — `focus()` , `scrollTo()` . Also the sanctioned way to talk to non-React surroundings, which a SharePoint page is.

4 · TypeScript & modern JS — rapid fire

Typing component APIs?

Discriminated unions for variant props (`kind: 'link' | 'button'` with the fields each branch requires) so illegal combinations don't compile; generics for data-driven components (`Table<Row>`), constrained with `extends` . Type the events precisely and let inference do the rest.

unknown vs any — and satisfies?

`unknown` forces narrowing before use — the honest type for API responses; `any` switches the compiler off and spreads. `satisfies` validates a value against a type while keeping its literal inference — ideal for config and token maps.

Utility types you actually use?

`Pick` , `Omit` , `Partial` , `Record` , `ReturnType` weekly; mapped types for theme tokens or deriving handler maps from a props map. The point to land: types encode intent, and good ones make invalid states unrepresentable.

Async model — the one-breath version, then what matters in UI code.

The event loop drains microtasks (promise callbacks) between macrotasks (timers, I/O); `async/await` is promise sugar with sane try/catch. What actually matters in components: cancel stale requests — `AbortController`, aborted in the effect cleanup — and guard against out-of-order responses with a last-write-wins check. Or let React Query own that whole class of bug.

ESM, tree-shaking, bundle size?

Named imports from ESM builds, honest `sideEffects` flags, and suspicion of barrel files that drag half a library in. On an intranet, every user loads the same web part daily — often over VPN — so bundle size is a first-class requirement, not a nicety.

Immutability in practice?

Spread is shallow — nested updates need care; `structuredClone` for real deep copies; immer when reducer updates get gnarly. React's

change detection is reference-based, so accidental mutation is how "it doesn't re-render" bugs are born.

5 · Jest + React Testing Library

The one skill the ad calls *essential*. Expect specifics, possibly a live exercise. Know the philosophy cold and one worked example by heart.

RTL's philosophy — and why does query order matter?

Test what the user experiences, not implementation internals, so refactors don't break green tests. The query priority mirrors that: `getByRole` with an accessible name, then `getByLabelText`, `getByText`, and `getByTestId` only as a last resort. Side benefit: role queries fail on inaccessible markup, so every test doubles as an accessibility smoke check.

fireEvent vs user-event?

`user-event v14` (`userEvent.setup()`, awaited calls) simulates the real interaction sequence — pointer, focus, keyboard — while `fireEvent` dispatches a single synthetic event.

Components genuinely depend on those sequences (blur validation, focus management), so `user-event` is the default; `fireEvent` only for edge cases it can't express.

getBy / queryBy / findBy?

`getBy` throws when missing — assert presence.
`queryBy` returns null — assert absence. `findBy` returns a retrying promise — assert async appearance; `waitFor` for arbitrary async assertions. Most "flaky" React tests are a `getBy` that should have been a `findBy`.

What is an `act()` warning telling you?

That an update happened outside the test's awaited flow — you asserted before React finished. The fix is almost never wrapping code in `act` manually; it's awaiting the user-event call, using `findBy`, or advancing fake timers properly.

Your mocking strategy?

A ladder: don't mock what you can render; mock the *network* with MSW so tests stay realistic and handlers get reused in Storybook and dev; `jest.mock` for genuinely heavy modules; spies to observe calls; fake timers for debounce and polling — with user-event's `advanceTimers` option so the two don't fight. Over-mocking is how suites end up testing their own mocks.

Custom hooks?

`renderHook` from RTL for reusable logic hooks. If a hook serves one component, I test through the component — that's where the contract lives.

Providers everywhere — how do you keep tests clean?

A custom render in `test-utils` wrapping the tree in the app's providers — theme, query client, i18n, a mocked SPFx context — with per-test overrides. One import change and every test gets a realistic environment.

Snapshots?

Low value as a default: they rot and people approve diffs blind. Small inline snapshots of stable serialized output are fine; visual appearance belongs to a visual-regression layer — Storybook + Chromatic — not Jest.

What would you test in a shared component library?

The public contract per variant and prop; keyboard interaction and focus behavior; accessibility with `jest-axe` per state; controlled and uncontrolled modes — and nothing about internals, so consumers upgrade without ceremony. Coverage enforced on changed code in CI: a floor, not a goal.

How do you test SPFx code? (*differentiator answer*)

Keep the web part class a thin shell and put everything real into ordinary React components and services injected via props — then it tests exactly like any React code, with SharePoint/Graph/PnPjs access behind a small interface you mock in Jest. Framework-touching glue stays minimal; true end-to-end goes to Playwright against a dev site when the project warrants it.

Worked example to narrate: async SearchBox

Debounced input → fetch → results list. Five tests you can list from memory:

1. Renders with an accessible label — `getByRole('searchbox')`.
2. Typing + advancing fake timers fires exactly one request with the final term (assert in the MSW handler).
3. Shows a loading state, then `findByRole('list')` with results.
4. Error path renders a retry affordance.
5. A slow stale response never overwrites a newer one — resolve handlers out of order.

```
const user = userEvent.setup({
  advanceTimers: jest.advanceTimersByTime,
});
await user.type(
  screen.getByRole('searchbox'), 'invoice');
jest.advanceTimersByTime(300);
expect(await screen.findByRole('list'))
  .toHaveTextContent('Invoice policy');
```

6 · SPFx & SharePoint crash course

Your gap — study section, not Q&A. Half a day of reading plus the build plan at the end turns "read about it" into "built with it".

Mental model

SPFx is Microsoft's client-side extensibility model for SharePoint Online, Teams and Viva. Your code runs *in the page*, in the signed-in user's context, with no iframe — which is exactly why Microsoft wraps it in governance: app-catalog deployment, admin-approved API permissions. Think "sanctioned micro-frontends for Microsoft 365".

What you can build

- **Web parts** — page building blocks; the bulk of this role.
- **Extensions** — Application Customizers (page chrome: headers, footers, toolbars), Field Customizers (list column rendering), ListView Command Sets (toolbar and context-menu actions), form customizers.
- **ACEs** — Adaptive Card Extensions for Viva dashboards.

Toolchain & lifecycle

Node LTS → scaffold (Yeoman generator today; a new open-source **SPFx CLI** is in preview and set to replace it) → local dev via `gulp serve` into a workbench or live page (a new **debugging toolbar** is rolling out to replace the old local workbench) → `bundle` → `package-solution` → `.sppkg` → upload to the tenant or site-collection App Catalog → approve requested API permissions → add to sites and pages. Promotion through separate catalogs/tenants per stage is what the ad's "development, test and acceptance environments" maps to.

Versions — the fact to know cold (as of July 2026)

1.22 shipped December 2025; **1.23 went GA mid-April 2026** (bringing the CLI preview and form-customization improvements); 1.24 previews next. The headline constraint: **SPFx still pins React 17**. Microsoft has targeted React 18 support around mid-2026 without a confirmed date — so don't promise React 18 features to anyone.

SAY IT LIKE THIS

"First thing I'd check is the tenant's SPFx baseline — it pins my React and TypeScript versions, and it decides whether the React 18 migration is already on our table."

Getting data

SPHttpClient	SharePoint REST — lists, libraries, sites.
MSGraphClientV3	Microsoft Graph — users, mail, Teams, files.
AadHttpClient	Your own Entra-protected APIs.
PnPjs	The ergonomic wrapper most teams actually use; fluent API, batching, retry behaviors.

Graph and API scopes are declared in `package-solution.json` (`webApiPermissionRequests`) and approved centrally by an admin — **least-privilege scopes** is the phrase a financial client wants to hear. Tokens are brokered by the framework in the user's context; there is no place for client-side secrets, ever.

SharePoint data gotchas — classic interview bait

- **List view threshold:** 5,000 items per query. Filter on indexed columns, page results, or go through search/Graph — never table-scan a big list from the browser.
- **Throttling:** expect 429/503 with `Retry-After`; honor it with exponential backoff (PnPjs has behaviors for this) and batch requests where possible.

UI layer

Fluent UI v8 (`@fluentui/react`) is the long-time SPFx default; **v9** (`@fluentui/react-components`) is a rewrite on Griffel with a new theming model. Projects mix them during migration at a bundle-size cost. Pull theme tokens from the SharePoint context so web parts match the intranet instead of fighting it.

Sharing code across solutions

Two roads, one trade-off — this maps straight onto the ad's "shared component libraries": an **npm package** in a private registry gives independent versioning but duplicates bytes into every bundle; an **SPFx library component** deploys one copy to the tenant but couples deployments. Know both; §7 goes deeper.

Build-before-interview plan (≈ half a day)

1. Scaffold a web part — React template, TypeScript.
2. Render a Fluent UI list of items read from a SharePoint list via PnPjs.
3. One property-pane setting (the list name).
4. Two Jest/RTL tests with the data service mocked behind an interface.
5. Package the `.sppkg` and deploy to a trial tenant — the old free M365 developer sandbox now requires a Visual Studio subscription, so a 30-day Business trial tenant is the practical route.

After this you can truthfully say: *"I've built, tested and deployed an SPFx web part — production depth is what I'd build here."*

7 · Architecture & shared libraries

How do you design a component library API?

Small composable primitives over god-components; a consistent controlled/uncontrolled duality; theme tokens instead of hardcoded style; accessibility built in — roles, keyboard, focus. Storybook as the living contract and review surface. Semver discipline with changelogs and deprecation windows; breaking changes ship with codemods or at minimum migration notes.

The ad says "consuming or extending" a shared library — how do you extend without forking?

Wrap, don't fork: composition and slots or render props first, sanctioned styling hooks second, and upstream contributions when the gap is general. A fork is a maintenance-debt decision taken explicitly with the library's owners — never a default.

Several SPFx solutions sharing code — how?

The npm-package vs library-component trade-off from §6, plus a versioning strategy so five web parts don't lock-step deploy. Multiple web parts on one page are effectively micro-frontends: independent bundles, so watch duplicated dependencies and design each one to fail gracefully on its own.

8 · Security & accessibility

The financial-sector lens: both are contractual here, not aspirational.

Security

- React escapes rendered strings — the XSS holes are the escape hatches:
`dangerouslySetInnerHTML` (only ever with sanitized input — `DOMPurify`), attacker-controlled URLs (`javascript:` in href), and anything you inject into the DOM outside React.
- **Supply chain:** lockfiles, `npm audit` in CI, a Renovate/Dependabot cadence — a bank will ask how dependencies get patched, so have the answer as a process, not a hope.
- **Secrets:** none in a frontend bundle, full stop. SPFx's token brokering exists precisely so you never touch credentials; request least-privilege Graph scopes and be ready to justify each one at approval time.
- Expect change control — peer review, protected branches, signed-off releases through dev/test/acceptance. Present that as how you prefer to work, not as an obstacle.

Accessibility

- Target **WCAG 2.1/2.2 AA**; in the EU institutional world EN 301 549 and the European Accessibility Act (in force since June 2025) make this a legal requirement.
- Semantics first, ARIA second; full keyboard operability with visible focus; focus trapping and restoration in dialogs; contrast.
- Testing: `jest-axe` per component state, RTL role queries as the everyday guard, and a periodic manual screen-reader pass. Fluent UI gives good defaults but doesn't absolve composition mistakes.

9 · Ways of working & STAR bank

Explaining trade-offs — their explicit requirement

Use one fixed frame every time: **options** → **constraints** (time, risk, maintainability, cost) → **recommendation** → **what we give up** and the trigger for revisiting.

REHEARSED SENTENCE

"Both work; A ships two weeks sooner, B halves the maintenance surface — given the acceptance deadline I'd take A now and schedule B's refactor behind a feature flag."

Sprint reality to convey

Refinement with BAs, honest estimates, and a Definition of Done that already includes tests, accessibility and documentation — so the ad's "support testing, validation, bug fixing" and "handover materials" sound like your normal, not extra work. Mention that you keep a lightweight decision log so handovers survive you leaving.

Five stories, pre-loaded (STAR, 60–90s each)

1. **Figma → production.** High-traffic platform feature with designers and BAs; the spec gaps you resolved; shipped behind Flagsmith flags. *(Collaboration + iterative delivery.)*
2. **Measured optimization.** GraphQL data-layer integration cutting payloads ~30% on targeted flows; separately, tooling that contributed to a ~10% smaller production bundle. *(Performance with numbers.)*
3. **End-to-end ownership.** Customer-support chat rebuilt as a horizontally scalable WebSocket service — Redis pub/sub, JWT — cutting infrastructure cost ~75% and p95 latency ~30%, owned end to end. *(Ownership; proves you understand the backends your UI calls.)*
4. **Scale awareness.** A *separate* system: the event broadcaster serving ~50K concurrent clients at ~100K messages/second, where your redesign work sped delivery ~10%.
5. **Mentoring & voice.** Onboarding, pair programming, code reviews; internal talks on AI, neural networks and TypeScript. *(Seniority beyond code — and it feeds the Neo'vations culture point.)*

DISCIPLINE

Stories 3 and 4 are **two different systems**. Keep them distinct under pressure — merging them into one mega-story reads as inflation the moment anyone probes.

10 · The gap questions — scripts

"DO YOU HAVE SHAREPOINT EXPERIENCE?"

"Production SharePoint, no — and I won't pretend otherwise. What I've done is ramp the SPFx model: built and packaged a web part reading list data through PnPjs, tests included. My recent years were production React on exactly the version SPFx runs. The platform specifics are the learnable part — the React, testing and enterprise-delivery depth I bring on day one."

Only say the "built a web part" part after actually doing §6's plan. You're one follow-up question away from being checked — and honesty here is also what keeps every later round consistent.

"YOUR LAST TITLE SAYS BACKEND — ARE YOU CURRENT ON FRONTEND?"

React at Docler is professional experience across the recent years, and your latest hands-on challenge was React 17 + TypeScript with tests. Have one concrete, recent UI decision ready to narrate — a state-shape choice, a memo boundary, a form architecture.

"HOW'S YOUR FRENCH?"

Luxembourg financial environments often run French socially and English technically. If asked: your honest level, actively improving, and note the delivery language of this ad is English. Better: ask *them* the working-language split proactively (§12) — it defuses the topic before it becomes one.

11 · The technical test stage

Their process runs "technical tests" before the business-manager round. Likely formats: a live React exercise, a Jest/RTL exercise, or a quiz.

- Restate the task and ask **one** clarifying question before typing.
- Narrate decisions — "controlled input so validation is live" — thinking aloud is the product being graded.
- Write, or at least name, the test first if the harness allows; keep the app running with small commits.
- When stuck, say what you'd look up. Seniors are allowed not to memorize; they're not allowed to flail silently.
- If AI tools are permitted, use them the way you do daily — and review the output loudly. Critically reviewing AI-generated code is itself a senior signal in 2026. Your own lesson applies: keep diffs minimal and match the repo's existing formatter, so reviewers see one intended change instead of six formatting ones.

12 · Questions to ask them

Pick four or five; each one signals seniority on its own.

- "Which SPFx version is the tenant on — and is the React 18 migration on your roadmap once Microsoft ships support?" (*Instantly signals you know the platform's core constraint.*)
- "Who owns the shared component library I'd be consuming and extending — the client or NEOFACTO — and how are breaking changes handled?"
- "What's the testing bar in CI — coverage gates, jest-axe, visual regression?"
- "How does a change travel dev → test → acceptance → production, and who approves App Catalog deployments?"
- "How is the squad composed — am I embedded with the client's team, and what's the working-language split day to day?"
- *For the recruiter stage:* "What does the exclusivity agreement cover in scope and duration?"

13 · Last 48 hours

- **Build the demo web part** (§6 plan) — half a day, the highest-ROI item on this list.
- Re-read §3 and §5 **out loud** — spoken answers, not silent recognition.
- Rehearse the 2-minute pitch and the five STAR stories once each, timed.
- Prepare three things word-for-word: your salary number, the Docler exit line, the SharePoint honesty script.
- Skim NEOFACTO's site and LinkedIn that morning; mention something specific — Neo'vations, the Aricoma context — once, naturally.
- Logistics: charged devices, backup connection for video, quiet room, joined three minutes early. Esch-Belval if in person.

Good luck.